

1. Introduction

SupplyLink is a full-stack supply chain management system developed to simulate how real-world logistics and inventory systems operate in enterprises. The primary objective of this project is to efficiently manage suppliers, warehouses, products, and shipment processes in a structured and scalable manner. In real-world industries, suppliers provide goods which are stored in warehouses, and these goods are later distributed through shipments. Managing this flow manually leads to inefficiencies, delays, and errors. Therefore, this project was designed to automate and streamline the complete lifecycle of supply chain operations. The project was implemented step-by-step starting from core Java concepts such as collections, moving to database handling using JDBC, then building scalable APIs using Spring Boot, and finally integrating a user-friendly frontend using Angular. This gradual implementation helped in understanding not only the technologies but also how they are applied in real-world systems.

2. Project Development Approach (Day 1–Day 25)

The development of this project was carried out in a phased manner. During Day 1 to Day 3, the focus was on understanding the structure of the system by creating entity classes such as Supplier, Warehouse, and Product. These classes were designed to reflect real-world objects. For example, a supplier provides goods, warehouses store them, and products represent the actual items being handled. On Day 2, collections such as ArrayList were used to simulate data storage. This allowed testing of logic such as adding suppliers, retrieving them, and sorting them without involving a database. On Day 3, JDBC was introduced to store the data permanently in a MySQL database. DAO classes were created to separate database logic from application logic. From Day 4 onwards, Spring Boot was introduced to convert the application into a REST API-based system. Controllers handled requests, services applied business logic, and repositories handled database operations using JPA. From Day 10 to Day 13, advanced modules such as shipment tracking and JWT-based authentication were implemented to make the system secure and enterprise-ready. From Day 14 to Day 25, Angular was used to build the frontend, where users could interact with the system through forms and dashboards.

3. Real-World Implementation Logic

In this project, the relationship between suppliers, warehouses, and products was implemented to reflect real-world supply chain flow. Each supplier is responsible for one or more warehouses. This means a supplier can manage multiple storage locations. Inside each warehouse, multiple products are stored. This relationship was implemented using One-to-Many mappings in JPA. For example, when a supplier is created, warehouses can be assigned to that supplier. Each warehouse has a capacity, and when products are added, the system checks whether the warehouse has enough capacity before storing them. This ensures realistic constraints are maintained. This structure allows clear segregation of suppliers and their respective warehouses. Products are always linked to a specific warehouse, ensuring proper tracking and management.

4. System Architecture Diagram

The architecture of the system follows a layered approach. Frontend (Angular) ↓ Controller Layer (Spring Boot) ↓ Service Layer (Business Logic) ↓ Repository Layer (JPA) ↓ Database (MySQL) The

frontend interacts with backend APIs through HTTP requests. Controllers receive these requests and pass them to the service layer. The service layer processes the logic and interacts with the repository layer, which communicates with the database.

5. Application Flow Diagram

The flow of the application is as follows: User → Register/Login → JWT Token Generated ↓ User sends request with token ↓ Controller validates request ↓ Service processes logic ↓ Repository interacts with database ↓ Response sent back to frontend This flow ensures that every request is authenticated and processed securely.

6. Entity Relationship Explanation

The core entities in the system are Supplier, Warehouse, and Product. Supplier → Warehouse → Product A supplier can have multiple warehouses. Each warehouse can store multiple products. This relationship ensures proper hierarchy and organization of data. This structure was implemented using JPA annotations such as OneToMany and ManyToOne, allowing efficient data retrieval and maintaining referential integrity.

7. Service Layer Role

The service layer plays a critical role in implementing business logic. It acts as an intermediate layer between controllers and repositories. For example, when adding a product, the service checks whether the warehouse exists and whether it has sufficient capacity. Only after validation does it allow the product to be stored. This ensures that the application follows business rules and prevents invalid data from being stored in the database.

8. Angular Frontend Implementation

The frontend was developed using Angular, where components were created for managing suppliers, warehouses, and products. Forms were implemented using reactive forms, allowing validation of user input such as email format and password strength. Angular services were used to send HTTP requests to the backend APIs. The frontend provides dashboards for users to view and manage data dynamically, ensuring a smooth user experience.

9. Conclusion

The SupplyLink project demonstrates a complete development lifecycle, starting from core Java concepts and progressing to a full-stack enterprise application. It integrates collections, JDBC, Spring Boot, JPA, JWT security, and Angular frontend into a single system. This project provides a strong understanding of how real-world applications are designed, developed, and deployed. Overall, this project enhanced practical knowledge and prepared for real-world software development and technical interviews.

